Muhammad Jehanzaib

AI Case Studies — Agents, Automation & Voice

In-depth engineering case studies of products I designed and built, grouped by domain. Each one covers the problem, the requirements, the architecture, the key decisions, and measured results.

mjehanzaib99@gmail.com · linkedin.com/in/muhammad-jehanzaib-4036b5195 · github.com/mjehanzaib999 · Resume: mjehanzaib-ai-resume.up.railway.app · Portfolio: mjehanzaib-ai-portfolio.up.railway.app

LIVE deployed & clickable · DEMO video walkthrough · FLAGSHIP lead production work

Index

Click a case study to jump to it. Each case study follows the same layout: at a glance → numbers → problem → requirements → architecture → how it works → decisions → results.

01 · FinTech / TaxTech

1.1 Ventive — AI R&D Tax Credit Platform FLAGSHIP DEMO — Contracts · documents · calculation · delivery

02 · Conversational AI / Telecom

2.1 Omvi William — Multi-Tenant Agentic AI SaaS with Real-Time Voice FLAGSHIP DEMO — Chat · voice · agent teams

01 · FinTech / TaxTech

1 case study

1.1 · Ventive — AI R&D Tax Credit Platform FLAGSHIP DEMO

FinTech / TaxTech

Client: a US R&D tax-credit advisory firm, delivered through Ventive · **Role:** lead AI engineer across six services, from architecture and PRDs to code, review and rollout · **Timeline:** 2026, multi-release and ongoing · **Demo:** Contract analysis walkthrough ↗

A platform that takes a tax firm's engagement from raw client documents to a signed, client-ready credit letter. AI agents read the contracts, extract the tax documents, compute the credit and fill the deliverables. Analysts approve at each gate, and the system learns from their corrections.

THE PROBLEM

- Every client contract read by hand to decide if the research was "funded".
- Payroll, ledgers and returns keyed into Excel workpapers manually.
- Credit computed, forms filled and letters written per client, hours each.

WHAT I BUILT

- A 13-node agent pipeline that decides each contract with evidence.
- Form-aware extraction for W-2, 1099, K-1, ledgers and census files.
- A rules-driven credit engine that fills workpapers, forms and the letter.

THE OUTCOME

- Documents in Drive flow to a filled workpaper with no one touching them.
- Contract verdicts agree with the firm's own answers ~86% of the time.
- Cost per analysis cut by a third; a 51-minute extraction now takes 7.

By the numbers

6 services behind one admin app and one shared schema	**13** nodes in the contract-analysis agent graph
−35% cost per analysis from prompt caching, verified like-for-like	**51 → 7.4 min** 906-employee census extraction, full coverage (was 676 rows)
~86% agreement with the firm's ground truth on a 38-contract replay	**to the cent** workpaper fills tie out to the source ledger
26 lifecycle events on the automation bus	**9,000+** automated tests across the repos

The problem

An R&D tax credit study is mostly reading and arithmetic done by expensive people.

- **Contracts:** each one must be read to decide whether the customer "funded" the research, which removes it from the credit.
- **Documents:** payroll, ledgers and prior returns arrive as scans, spreadsheets and PDFs and are keyed into Excel by hand.
- **Calculation and delivery:** the credit is computed under several methods, filled into Form 6765 and state forms, and written up as a letter.
- **Consistency:** each step took hours per client and the answer depended on which analyst did it.

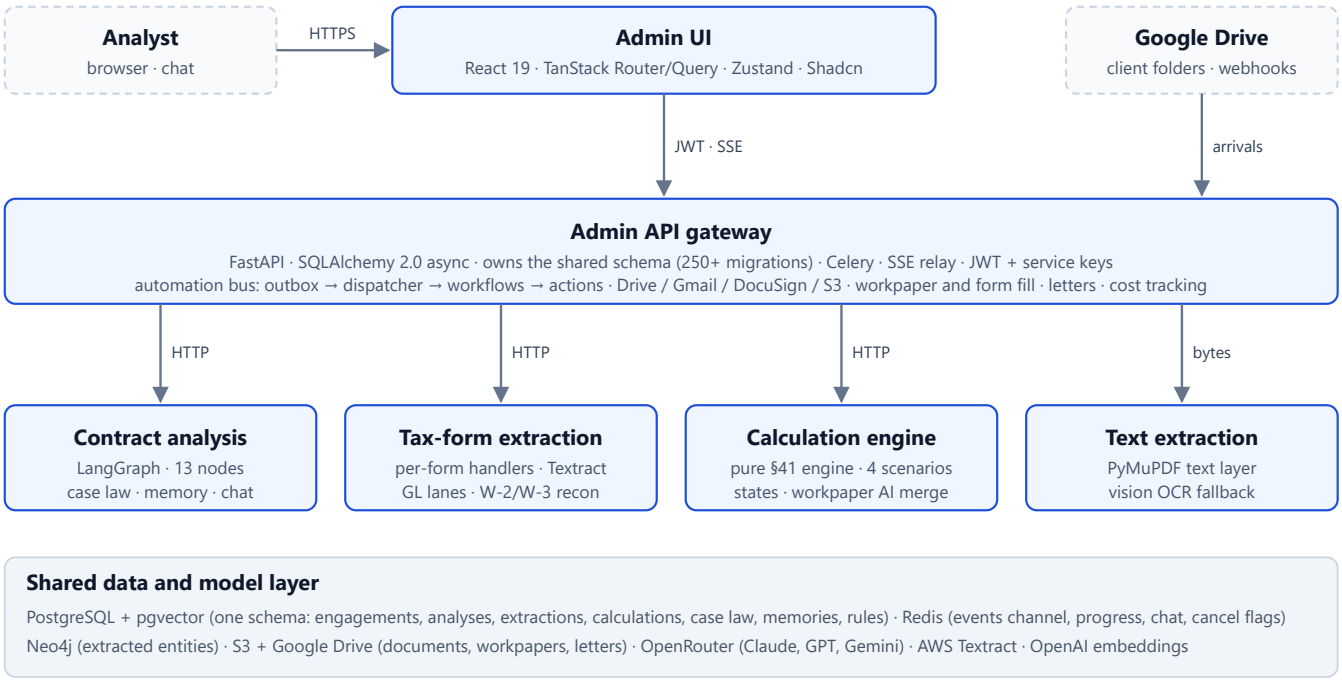
The firm wanted the whole chain automated, with a human approving at the points where judgement or liability sits.

Requirements I derived and built against

Requirement	How the system meets it
Decide "funded research" per contract, with evidence	A 13-node LangGraph pipeline: document check, triage, web search, a rights test and a payment-risk test in parallel, adversarial self-review, an IRS counter-argument step, and a merge that derives verdict, confidence and tax position.
Never guess when the contract is silent	Triage questions pause the run; the analyst answers in chat and the run resumes from a cached first pass instead of re-paying the whole pipeline.
Ground decisions in law	A case-law and state-conformity knowledge base in Postgres with pgvector. Mandatory cases are always injected; the rest are retrieved by similarity per node.
Handle hundreds of contracts at once	Batch upload, deterministic bundling of related files (MSA + change orders, master PDF + SOW) with an LLM fallback, analyst bundle review, paced fan-out, and cancel that propagates to every worker.
Read every tax document the firm receives	A per-form handler registry (W-2/W-3, 1099, K-1, 2553, 6765, 1120/1065, census, general ledger), AWS Textract for scans, an LLM path for text, dual-model consensus, and W-2 ↔ W-3 reconciliation per EIN and year.
Read any ERP's general ledger	An LLM head-verdict detects a ledger, a layout-inference step maps columns, and seeded dialects (QuickBooks, SAP B1) parse postings with a running-balance check.
Compute the credit correctly and explainably	A pure engine: 4-scenario matrix (ASC / regular × with and without §280C), controlled-group allocation, Decimal money, full audit trail, rule-driven recommendation. Rules are versioned data per jurisdiction and year.
Fill the firm's own Excel workpapers	Header-aware sheet mapping plus an AI merge agent. Every written value must trace to the source extraction; one invented value fails the whole fill. Gaps are completed from the ledger itself.
Produce the deliverable	Government forms filled from the same data, a branded Credit Release Letter with draft/final lifecycle and Drive archival, email delivery, and DocuSign behind a kill-switch.
Run itself as documents arrive	A transactional outbox turns 26 engagement, study and interview transitions into events. Admin-authored workflows bind events to actions with dedupe, cascade guards and a run audit dashboard.
Learn from analysts	An accepted correction is distilled into a lesson; the analyst approves or edits it; later runs retrieve approved lessons per node by similarity and show which ones they used.
Show cost and progress	Token and dollar cost per node, live per-stage progress, re-run records stamped at dispatch, and a stall watchdog that fails wedged runs closed.

Architecture

One admin API owns the shared Postgres schema and every integration. The three AI engines are separate services that never write each other's tables. Requests go over HTTP; completions come back on one Redis event channel, so the gateway can fan out, wait, reconcile and re-drive without any engine knowing about the others.



Blue boxes are services I built or led; dashed boxes are external. Completions flow back on one Redis channel; the gateway correlates them to workflow runs, reconciles misses on a timer, and relays them to the browser as live progress.

The six services

- **Admin UI** (React 19, TypeScript): engagement timeline with live progress, batch upload and bundle review, analysis reports, an agentic chat for correction re-runs, editors for rules, prompts and memories, workflow authoring, run audit, letter and form cards.
- **Admin API gateway** (FastAPI): the only writer of the shared schema. Orchestrates batches, hosts the automation bus, fills workpapers and forms, renders letters, fronts Drive, Gmail, DocuSign and S3.
- **Contract analysis engine** (FastAPI, LangGraph, Celery): the agent graph, per-node case-law and memory retrieval, pause and resume, streaming chat with tools, correction distiller, stall watchdog, per-node cost events.
- **Tax-form extraction** (Python, Textract, LLM): classifier, per-form handlers, ledger and census lanes, reconciliation, Neo4j structuring.
- **Calculation engine** (Python, FastAPI, Celery): pure compute with a thin IO shell, readiness checks and claims so a fill runs once per input, AI merge agents with source guards.
- **Text extraction** (FastAPI): classifies each PDF as text, image or hybrid and routes to native text or vision OCR, with warnings that gate low-quality input.

How it works

A contract, start to finish

1. **Upload and bundle.** Files are grouped into contracts by filename pattern, then by an LLM for the ambiguous ones. The analyst confirms the bundles and picks the taxpayer.
2. **Document check.** The engine extracts eight verbatim clause families (IP, payment, termination, governing law, incorporated clauses and more), folding in clauses incorporated by reference.
3. **Triage.** If the contract is silent on something that matters, the run pauses with questions. The analyst answers in chat and the run resumes.
4. **Two prongs in parallel.** A rights test (who owns the research) and a payment-risk test (who bears the cost of failure), each grounded in retrieved case law.
5. **Challenge.** An adversarial review argues the other side; an IRS counter-argument step writes what an examiner would say.

6. **Verdict.** One derived-verdict step merges the prongs into qualifies / does not qualify, a confidence score and a tax position. It is the only node allowed to write the verdict.
7. **Review and learn.** Low-confidence results wait for review. A correction re-runs only the affected nodes, and an accepted correction becomes a lesson for future runs.

A document, start to finish

1. **Arrival.** A file landing in the client's Drive folder emits an event. Content-hash dedupe stops re-uploads from running twice.
2. **Plan.** The gateway works out what the engagement still needs (which forms, prior years, states) from the rule bundle and submits only those.
3. **Extract.** Each chunk is classified and routed to its form handler. Multi-page and multi-copy forms are merged. Ledgers go through layout inference and a balance walk.
4. **Reconcile.** W-2 totals are checked against the W-3 per EIN and year with cents-level tolerance.
5. **Fill.** When the inputs are complete, the calculation service claims the fill and the AI merge agent writes the workpaper rows, every value checked against its source.
6. **Calculate and deliver.** After workpaper approval, the engine computes the scenarios, the analyst accepts or overrides, and forms and the letter are generated and sent.

Agent engineering

- **Schema-constrained outputs:** every node returns strict JSON against a wire schema with a parse backstop. A malformed answer is retried once per failure kind, never silently accepted.
- **Selective re-runs:** node outputs are captured at completion. A correction marks the nodes it touches dirty and re-runs only those and their consumers, derived from what each node reads.
- **Prompt caching:** the contract is sent once as a cached block and reused by the six nodes that read it. Moving response schemas off the wire kept the cache key stable. Same contract: $3.31 → $2.16 per run.
- **Reasoning tiers:** mechanical nodes run at low effort, analytical nodes at medium. Output tokens fell 22% and wall-clock 21% with an identical verdict.
- **Safety rails:** abort probes with heartbeats between nodes, a 10-minute stall sweep that fails closed on Redis outage, publish-after-commit so no event precedes its data, and scrubbing of model text that could forge a prompt frame.
- **Correction memory:** lessons are embedded before they are saved, retrieved per node by cosine similarity (pinned lessons always), and stamped on each run so the report shows which memories were considered and applied.

Human in the loop

The system pauses where judgement sits, and every gate leaves an audit row:

- Triage questions before analysis, and bundle review before dispatch.
- A pending-review state for low-confidence verdicts.
- Workpaper approval before calculation; accept or override the recommended scenario with a required reason.
- A five-minute chat card to approve, edit or reject each distilled lesson.
- A reviewed-narrative gate before a letter can be finalized.

Key engineering decisions

Problem	Decision
Three AI services sharing one database	The gateway owns all migrations; engines declare foreign-owned tables and only write their own. Fresh-DB replay of the full chain is a release gate.

Verdicts written by four different nodes	Remove the fork so both prongs always run; one derived-verdict step is the only author of determination and confidence.
Completion events lost or duplicated	Redis pub/sub for speed plus a timer reconcile and a dispatch marker; one unified writer delivers the result whether it arrived live, on reconnect or on sweep.
A crashed re-run that looked like a 10-hour "Analyzing"	Re-run records stamped at dispatch, heartbeats per node, a stall sweep with compare-and-set, progress stored as a Redis hash so parallel branches cannot clobber each other.
LLM filling a spreadsheet invents or collapses rows	Every written value must trace to the source; recurring identical postings are kept distinct by count; any deficit is completed from the ledger deterministically.
Retries multiplying across layers	One retry ticket per failure kind threaded through every layer; worst case went from 18 billable generations to 4.
Tax rules changing every year	Rules, prompts and templates are versioned data in a knowledge base with an editor and a sign-off gate, not code.
Learning without retraining	Distill corrections into short lessons, gate them on human approval, retrieve by similarity per node, stamp each run with the lessons it used.

Results

Measure	Result
Sandbox replay of a real client year (38 contracts)	**31 completed, 7 for review, 0 failed**; 38 of 38 workpaper rows bound; ~86% agreement with the firm's ground truth
Cost per contract analysis (same contract, same model)	**$3.31** → **$2.16** with prompt caching (−35%); output tokens −22% with reasoning tiers
906-employee census extraction	**51 min** → **7.4 min**, 906 of 906 rows (was 676)
Expense workpaper from a general ledger	**105 rows**, contract-research total tied to the cent
Arrival-driven pipeline, end to end	file arrival → extraction → readiness → calculation → workpaper in Drive, **unattended**

- The extraction-to-workpaper chain ran unattended on a second client's 24 documents, including 9 stranded runs auto-redriven by the sweep.
- The correction-memory loop was closed live: an approved lesson was applied by three nodes on the next similar contract and flipped the verdict correctly.
- Delivered as stacked, reviewed pull requests across six repositories, each with CI: lint, type-check, migration replay and full test suites.

Python · FastAPI · LangGraph · Celery · Redis · PostgreSQL + pgvector · SQLAlchemy 2.0 async · Alembic · Neo4j · AWS Textract · PyMuPDF · pypdf · openpyxl · OpenRouter (Claude Opus / Sonnet, GPT, Gemini) · OpenAI embeddings · React 19 · TypeScript · Vite · TanStack Router/Query · Zustand · Shadcn UI · Google Drive & Gmail APIs · DocuSign · S3 · Docker · GitHub Actions

Watch demos: Contract analysis · Tax-form extraction · 1 | 2 | 3 | 4

02 · Conversational AI / Telecom

1 case study

2.1 · Omvi William — Multi-Tenant Agentic AI SaaS with Real-Time Voice

FLAGSHIP DEMO

Conversational AI / Telecom

Client: World Mobile (telecom operator), delivered through Omvi AI · **Role:** architect and engineer, designed and built end to end · **Timeline:** Oct 2025 – Mar 2026, demo platform Oct 2026 · **Demo:** Watch the walkthrough ↗

A white-labelled AI assistant, "William," that people can talk to or type to. It answers from the operator's own knowledge base, acts in email and calendar, remembers earlier conversations, and turns a voice call into a project that a team of five AI agents completes while the user watches.

THE PROBLEM

- One assistant for customers and staff, by voice and by text.
- Answers from the company's own material, not the open web.
- Fast enough on a live call that it never feels like a bot.

WHAT I BUILT

- One LangGraph brain behind both chat and voice.
- Six services: gateway, graph server, voice agent, RAG, orchestrator, Studio.
- Multi-tenant from day one: 6 roles, scoped state, cross-org guard.

THE OUTCOME

- Agent starts speaking 2.0 s after the question ends (median).
- A call becomes up to 20 tasks that five agents finish in 15–20 s each.
- Sellable to the next client without a rebuild.

By the numbers

2.0 s
end of question → agent starts speaking (median, 9 live calls)

3.9 s
end of question → grounded knowledge answer starts

35 → <10 s
first reply after a cold start

15–20 s
per task through all five agents

6
services around one agent brain

15 + 6
LangGraph nodes in the chat and voice graphs

25
agent tools (12 email, 7 calendar, 6 general)

6
RBAC roles in the multi-tenant model

The problem

- **One assistant, two channels:** customers and staff should get the same answers whether they talk or type.
- **Grounded, not generic:** answers must come from the operator's own knowledge base, and the assistant must say so when it finds nothing.
- **Act, not just answer:** email, calendar and tasks, with confirmation before anything is sent.
- **Feel instant on a call:** long silences make a voice assistant feel broken.
- **Sellable again:** the vendor, Omvi AI, needed the product to serve the next telecom or enterprise client without rebuilding it.

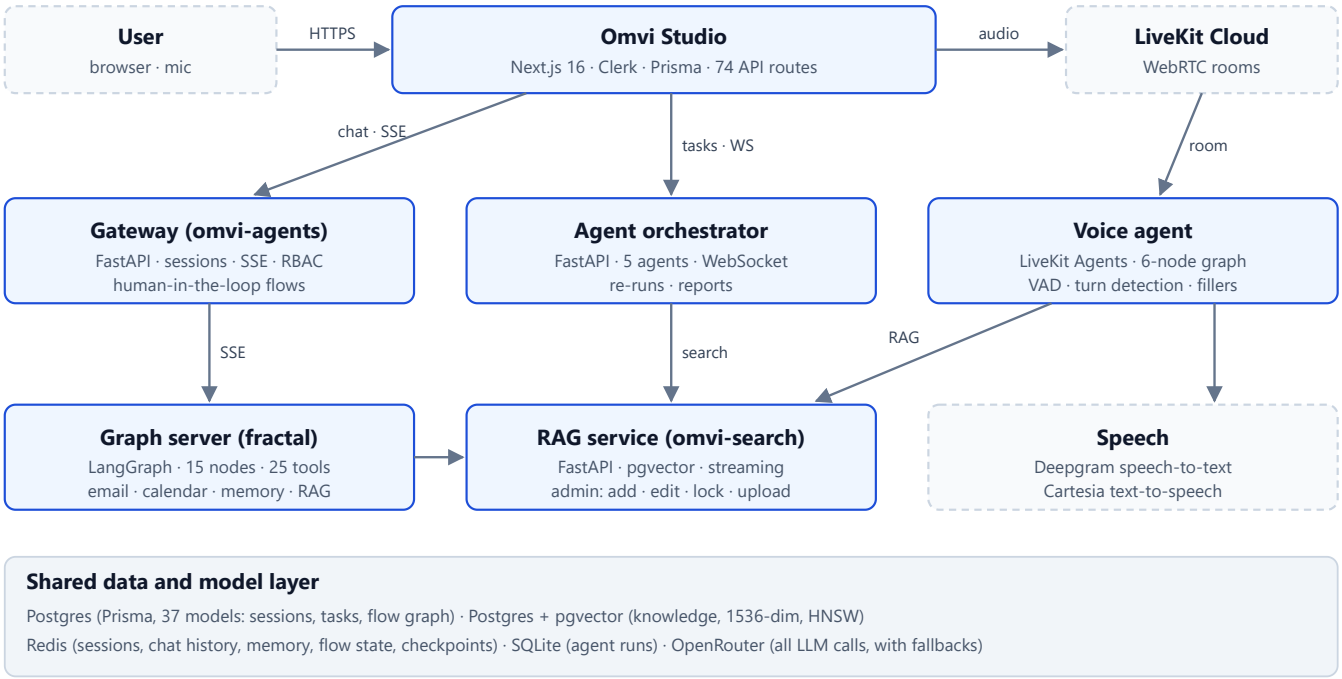
Requirements I derived and built against

Requirement	How the system meets it

Talk or type, and get the same answers	One LangGraph agent behind both: chat streams over SSE through the gateway; voice runs the same reasoning inside a LiveKit worker.
Answer from the operator's knowledge, not the web	pgvector RAG over the company corpus with neighbouring-passage context and a similarity floor; says so plainly when nothing relevant is found.
Take actions in email and calendar	Dedicated intent resolvers with 12 email and 7 calendar tools over Google and Microsoft OAuth; drafts are confirmed before anything is sent.
Remember the user	Temporal and cross-session memory recall nodes over Redis chat history, with safeguards against fabricated recall.
Guide new users	Human-in-the-loop onboarding and daily-reminder flows that pause for the user with LangGraph `interrupt()` and resume where they left off.
Turn conversation into work	A call transcript becomes a titled session with up to 20 tasks across 8 lanes and their dependencies; five agents then complete each task.
Feel instant on a phone-style call	Under 2.5 s from the end of a question to the agent speaking, measured on the deployed system.
Serve many clients from one codebase	Organization-scoped state, 6 roles mapped to `module:action` scopes, per-endpoint scope checks and a cross-organization guard.
Let admins control the knowledge	Knowledge entries can be added, edited, locked against changes, deleted, or uploaded as PDF, DOCX, HTML or text.
Show cost and progress	Per-agent output, timing and token cost; a live usage dashboard; notifications when agents finish.

Architecture

Six services, so the agent, the voice runtime and the task agents can each scale and ship on their own while sharing one brain and one data layer.



Six services (blue) around one agent brain; dashed boxes are external. Chat flows Studio → gateway → graph server; voice flows LiveKit → voice agent; tasks flow Studio → orchestrator, with progress pushed back over WebSocket.

The six services

- **Omvi Studio** (Next.js 16, Clerk, Prisma): the workspace. Sessions, chat, the voice call screen with live transcript, Board / Grid / Canvas task views, email and calendar cards, dashboard, knowledge admin, notifications, search.
- **Gateway** (FastAPI): the single stateful entry point. Redis sessions and history, SSE streaming, role and scope checks, LangGraph flows that pause for user input.
- **Graph server** (FastAPI + LangGraph): the agent brain, split out so it ships independently. Streams tokens back to the gateway over SSE.
- **Voice agent** (LiveKit Agents): joins each call room, listens, reasons with its own graph and speaks; kept warm so it joins quickly.
- **RAG service** (FastAPI + pgvector): retrieval and grounded answers, streamed or whole, plus the knowledge admin API.
- **Agent orchestrator** (FastAPI): watches for new tasks and runs each through five agents, pushing progress to the Studio over WebSocket.

How it works

A chat turn

1. **Route.** A main orchestrator node decides the turn. Email and calendar requests go to their own decision → intent resolver → action chains; everything else passes an LLM domain classifier.
2. **Ground.** Knowledge questions retrieve from the RAG service; personal questions hit temporal or cross-session memory recall.
3. **Answer.** A tool-calling speaker node writes the reply, bound to knowledge search, web search, time and task creation. Only answer-writing calls stream to the user; routers stay internal.
4. **Pause when needed.** Onboarding, daily-reminder and content flows stop with `interrupt()`, checkpoint in Redis, and resume days later with `Command(resume=…)`.

A voice call

1. **Join.** A warm worker joins the LiveKit room with no cold start. Silero VAD, a multilingual turn detector and noise cancellation handle the audio.
2. **Listen.** Deepgram transcribes; a six-node graph (decide, greeting, general, memory recall, retrieval, content) handles the turn.
3. **Fast route.** A clear product question skips the LLM router and goes straight to retrieval.
4. **No dead air.** A short filler line is spoken the moment retrieval starts; the answer then streams sentence by sentence through Cartesia.
5. **Call to project.** On hang-up the transcript becomes a titled session with 1–20 tasks, lanes and dependencies.

From tasks to finished work

1. **Pick up.** The orchestrator takes new tasks as they are written and also polls the task table every 3 seconds, because voice-created tasks never call its API.
2. **Run.** Up to three tasks at a time go through Omni (research), Planner, Builder, Reviewer and Nexus (write-up).
3. **Show.** Each stage streams lane transitions to the Studio and stores output, timing and tokens; the last stage produces a downloadable report. Refine re-runs with a user instruction instead of starting over.

Agent engineering

- **Streaming end to end:** on Python 3.10 nested LLM calls only reach the graph's event stream when a node hands down its run config, so each answer node passes it explicitly; the RAG service's own stream is relayed through the graph as custom events.

- **Graph controls speech:** I patched the LiveKit LangChain adapter so the graph, not the plugin, decides what is spoken.
- **Resilience:** every model call goes through OpenRouter with ordered fallback models and retries, so a rate-limited provider becomes a slower answer, not an error.
- **Knowledge:** documents chunked to about 1,800 characters, embedded with `text-embedding-3-small` into Postgres with an HNSW cosine index. Each query retrieves at least 8 passages above a similarity floor plus the passage after each of the top four. Answers by Gemini 2.5 Flash Lite with GPT-4.1 mini and GPT-4o mini as fallbacks.

Multi-tenancy and platform

- **Tenant isolation:** every request carries an organization, role and scopes; Redis state is keyed `{namespace}:{org}:{user}:{session}`; a middleware guard blocks cross-organization access.
- **Roles and scopes:** six roles (platform admin through client viewer) map to `{module}:{action}` scopes, enforced per endpoint. I authored the architecture plans the team adopted for serving many clients from one codebase.
- **Delivery:** shipped to GCP through GitHub Actions, later as 8 services on Railway (six apps plus Postgres with pgvector and Redis) with auto-deploy on merge and Playwright browser audits before release.

Key engineering decisions

Problem	Decision
Chat and voice drifting apart	One LangGraph brain; the voice worker runs the same reasoning and a patched adapter lets the graph control speech.
Silence while the agent thinks	Skip the router for clear product questions, speak a filler immediately, stream the answer by sentence.
Answers arriving all at once	Pass run config into answer nodes, tag user-facing calls, relay the RAG stream as graph events.
Slow first request after deploy	Build and compile the agent graph at startup; first reply went from 35 s to under 10 s.
Provider outages and rate limits	One OpenRouter gateway with ordered fallback models and retries in every service.
Voice-created tasks invisible to the agents	Orchestrator watches the task table directly, in addition to its trigger API.
Each new client needing a fork	Organization-scoped state, role-to-scope permissions and per-endpoint checks from day one.

Results

Voice latency measured on the deployed system on 3 November 2026: nine live calls, each a spoken question played into a browser microphone, timed from the live transcript.

Moment	Median	Range
Click "Voice" → agent greets the user	**7.5 s**	7.4 – 10.1 s
End of a knowledge question → agent starts speaking	**2.0 s**	1.7 – 2.6 s
End of a knowledge question → grounded answer starts	**3.9 s**	3.5 – 4.5 s
End of a general question → answer starts	**4.0 s**	2.8 – 4.0 s

- All nine spoken questions were transcribed correctly and answered from the knowledge base, including exact plan prices.
- A task created from a call finishes all five agents in about 15–20 seconds; Refine re-runs in about 13 seconds.
- The first reply after a deploy dropped from 35 s to under 10 s.

Python · LangGraph · LangChain · FastAPI · LiveKit Agents · Deepgram · Cartesia · Silero · PostgreSQL + pgvector · Redis · SQLite · OpenRouter (Gemini 2.5 Flash Lite, GPT-4.1 mini, GPT-4o mini) · SSE · WebSockets · Next.js 16 · TypeScript · Prisma · Clerk · Docker · Railway · GCP · GitHub Actions · Playwright

Watch demo ↗

---

mjehanzaib99@gmail.com · linkedin.com/in/muhammad-jehanzaib-4036b5195 · github.com/mjehanzaib999